

Virtualizzazione e Integrazione di sistemi

Introduzione alla Sincronizzazione degli orologi

NTP

Vittorio Ghini

a.a. 2021/22

Contenuto del documento

- Obiettivi della sincronizzazione
- Tipi di clock: Hardware e Software
- Meccanismo base di sincronizzazione
- ~~Orologi Logici di Lamport~~
- Universal Coordinated Time - UTC
- Proprietà di Correttezza e Monotonia dei Clock
- Sincronizzazione di clock per sistemi asincroni
- ~~Algoritmo di Cristian per sincronizzazione interna~~
- ~~Algoritmo di Berkeley per sincronizzazione esterna~~
- Network Time Protocol - NTP
- ~~Sicurezza in NTP~~

Sincronizzazione degli orologi

Un sistema distribuito è costituito da un insieme di processi, operanti su computer diversi, che comunicano attraverso lo scambio di messaggi per realizzare una applicazione. Solitamente questi processi operano in modo autonomo (seppur coordinato) sfruttando l'orologio locale del computer e non hanno accesso ad un clock comune in quanto non necessitano di un riferimento temporale unico.

Lo scopo di sincronizzare due computer è assicurare che diversi processi fisicamente distanti (cioè su computer diversi) abbiano la stessa nozione del tempo.

La nozione di tempo richiesta può essere assai diversa.

- In particolare, la sincronizzazione può essere usata per stabilire una delle seguenti caratteristiche temporali:
 - **L'istante** in cui un evento si è verificato
 - **L'intervallo** di tempo fra due eventi
 - **L'ordine relativo** fra vari eventi (clock logici, es Lamport).

L'importanza della sincronia temporale

- In molti sistemi distribuiti comunemente utilizzati, i sistemi e protocolli deputati alla sicurezza fanno uso di marche temporali per concedere permessi di accesso con una validità limitata nel tempo. In tal modo si limita il periodo di tempo durante il quale si potrebbe subire attacchi.
- Tra questi protocolli e sistemi ricordiamo SSL/TLS e Kerberos.
- La sicurezza di Kerberos è basata anche sui time stamp (marche temporali) dei ticket perciò è d'importanza critica che gli orologi dei server kerberos siano regolati con accuratezza.
- In kerberos, i ticket hanno una scadenza breve per prevenire attacchi di forza bruta e attacchi di replica.
- Permettendo agli orologi di subire scostamenti si rende la rete vulnerabile a questi attacchi.
- A causa dell'importanza della sincronia degli orologi nella sicurezza del protocollo Kerberos, se gli orologi non sono sincronizzati entro un ragionevole intervallo Kerberos presenta errori fatali e smette di funzionare.
- I client che tentino di autenticarsi da una macchina con un orologio non accurato falliranno il tentativo di autenticazione presso il KDC a causa della differenza di ora con il suo orologio.

Tipi di Clock (1)

Esistono due tipi di clock in un computer:

1. il **clock hardware** (o **CMOS clock**) alimentato da una batteria
 2. il **clock software** (o **system clock**) che avanza solo con s.o. in esecuzione.
- **Digital Clock (o Hardware Clock) (o CMOS clock)**
 - E' realizzato mediante un contatore digitale alimentato da una batteria.
 - Conta il passare del tempo anche a computer spento.
 - All'accensione del computer il valore del clock hardware viene usato per configurare il valore del clock software (o clock di sistema).
 - E' sensibile a variazioni della tensione di alimentazione o della temperatura.
 - La frequenza è fissa o con variazioni limitate (L'elemento base per la generazione del clock fisico è un oscillatore al quarzo che produce un segnale con una frequenza abbastanza stabile).
 - E' possibile leggere il clock hardware ed anche modificarlo.
 - L'operazione di modifica (adjust) in Linux è svolta dal comando `hwclock`.
 - In Linux, quando è attivo un servizio di sincronizzazione del tempo via rete (ad es. NTP) il kernel ad intervalli regolari assegna al clock hardware il valore di orologio ottenuto via rete.

Tipi di Clock (2)

- **Software Clock**

- E' anche detto clock di sistema poiché viene calcolato e fatto avanzare dal sistema operativo (e quindi via software) a partire dall'orologio hardware. Avanza perciò solo a computer acceso e s.o. in esecuzione.
- Al boot, il s.o. assegna al clock software il valore del clock hardware.
- L'avanzamento avviene così:
- L'orologio hardware poco preciso scatta ad intervalli abbastanza regolari e produce l'incremento unitario di un contatore (tick).
- Il valore del contatore (tick) è convertito in un tempo generando il valore corrente del clock software.
- Durante la conversione, si può aggiungere/sottrarre un tempo di offset per ottenere la sincronizzazione.
- Ogni volta che il contatore viene incrementato (ogni volta che scatta il tick) viene aggiornato il valore dell'orologio software aggiungendo il tempo trascorso dall'ultimo tick. Questo valore aggiunto ad ogni tick non è costante ma può variare nel tempo se ci si accorge che occorre accelerare o rallentare l'orologio software. Il valore di questa aggiunta è ciò che deve essere determinato dagli algoritmi di sincronizzazione.
- Se il sistema operativo sincronizza il suo clock di sistema con delle sorgenti esterne, allora ad intervalli regolari il s.o. assegna al clock hardware il valore del clock software ottenuto da rete.

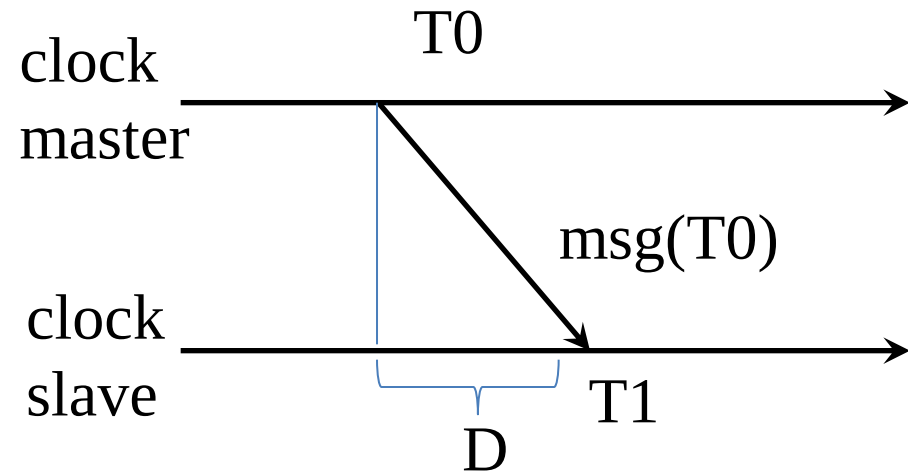
Meccanismo **banale** di sincronizzazione

- La sincronizzazione richiede la **comunicazione**.
- **Non esiste sincronizzazione senza comunicazione**
- Il sender (ruolo master) periodicamente invia un messaggio contenente il suo "Clock corrente" (Timestamp).
- Il ricevitore (ruolo slave) si sincronizza col Sender modificando il suo clock al valore del timestamp del messaggio ricevuto (funziona se la latenza è piccola rispetto alla precisione desiderata)



- Poiché il messaggio viaggia in rete, il ricevitore lo riceve con un certo ritardo e quindi riceve un valore di orologio relativo ad un istante passato (errore di fase).

- Assegnando, all'istante T_1 , il valore T_0 al clock dello slave si commette un errore D pari al tempo di trasmissione.
- Non si riesce a calcolare l'errore poiché T_0 e T_1 sono valori espressi da orologi diversi



Meccanismo banale di sincronizzazione

Miglioramenti

- Poiché il messaggio di settaggio del clock viaggia in rete, il ricevitore lo riceve con un certo ritardo e quindi riceve un valore di orologio relativo ad un istante passato (errore di fase).
- Per migliorare questo meccanismo aggiungiamo una fase preliminare in cui il sender calcola approssimativamente l'errore di fase misurando il **round trip-time** (tempo di andata e ritorno) tra sender e receiver, e lo fa inviando un messaggio e ricevendo la risposta dal receiver e calcolando la differenza tra i due tempi.



- Può funzionare, sotto certe ipotesi, anche se la latenza è elevata rispetto alla accuratezza desiderata.

Meccanismo **round-trip** di Sincronizzazione (1)

detto algoritmo di Berkeley

- I valori $T1$ e $T4$ sono espressi secondo l'orologio dello slave
- I valori $T2$ e $T3$ sono espressi secondo l'orologio del master

Meccanismo

- Lo slave spedisce il proprio clock $T1$ nel messaggio di richiesta.
- Il master risponde mettendo nel messaggio di risposta il valore $T1$ ricevuto, il valore $T2$ dell'istante di ricezione, il valore $T3$ dell'istante di spedizione della risposta che è anche il valore del clock dato allo slave.
- **Lo slave calcola il tempo presunto di trasmissione dal master allo slave approssimandolo con la metà del Round Trip Time**

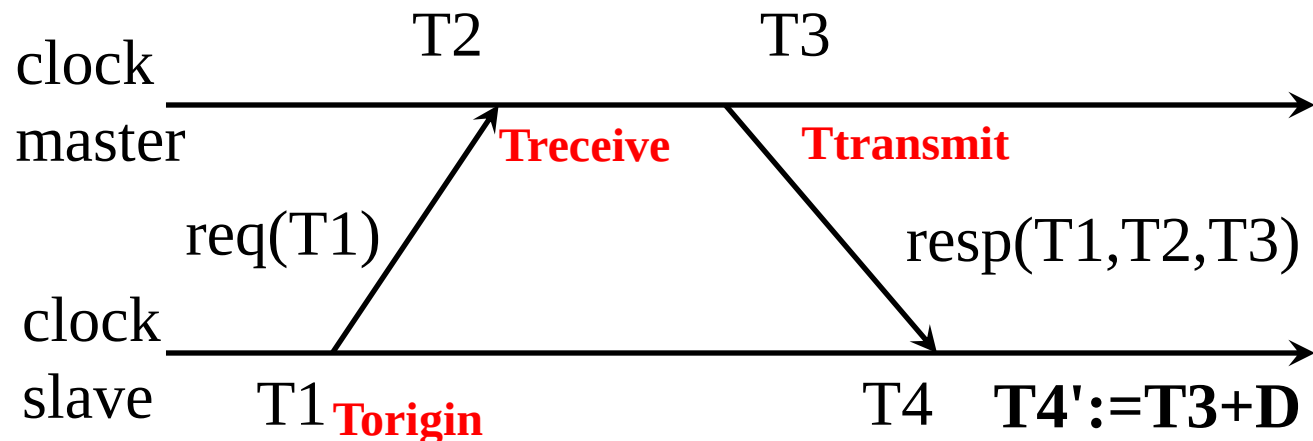
$$D = RTT/2 = ((T4 - T1) - (T3 - T2))/2$$

- Lo slave all'istante $T4$ cambia il proprio clock assegnandogli il valore

$$T4' := T3 + D = T3 + ((T4 - T1) - (T3 - T2))/2$$

SVANTAGGI:

Lo slave ottiene il clock, ma il master non sa quale è il clock dello slave e lo scostamento



Meccanismo **round-trip** di Sincronizzazione (2)

- È possibile stimare sia il delay di trasmissione da master a slave ed anche lo scostamento tra i due orologi.
- È una stima che approssima bene la realtà solo se:
 - i tempi di trasmissione si mantengano abbastanza costanti nel tempo.
 - il tempo di andata e il tempo di ritorno sono simili (non va bene per percorsi asimmetrici).
- Se sono valide queste condizioni allora posso stimare così Offset e Delay
- **Delay** $D \approx RTT/2 = ((T4-T1)-(T3-T2))/2$

Rappresenta il tempo che una richiesta impiega per raggiungere il server sommato al tempo che la risposta impiega per tornare indietro; la prima quantità della relazione rappresenta il tempo, relativo al client, tra la trasmissione della richiesta e la ricezione della risposta, mentre il secondo addendo rappresenta il tempo di elaborazione sul server della richiesta, prima che spedisca la relativa risposta.

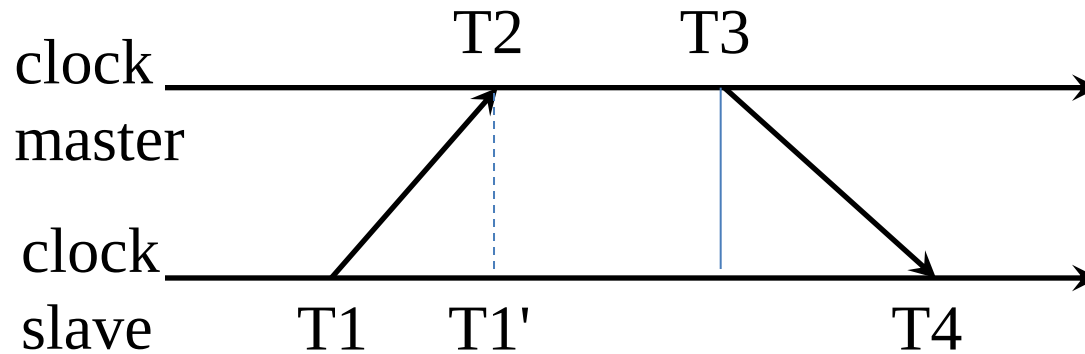
- **Offset** $O \approx ((T2-T1)+(T3-T4))/2$

Rappresenta la differenza di tempo tra due orologi, cioè è l'ammontare di tempo da sommare al proprio orologio per sincronizzarlo con quello della sorgente di riferimento. Consente di ottenere la sincronizzazione del proprio orologio.

Offset indica di quanti secondi bisogna rallentare il clock dello slave

Meccanismo **round-trip** di Sincronizzazione (3)

SPIEGAZIONE FORMULE



Offset è la differenza tra Tslave e Tmaster in uno stesso istante reale

$$\text{Offset} = T_{\text{slave}} - T_{\text{master}}$$

L'istanza T1 è approssimabile con $T1' \approx T1 + \text{RTT}/2$

dove $\text{RTT} = ((T4 - T1) - (T3 - T2))$

$$\begin{aligned} \text{Offset} &= T2 - T1' \approx T2 - (T1 + \text{RTT}/2) = \\ &= T2 - T1 - ((T4 - T1) - (T3 - T2))/2 = \\ &= 2 \cdot T2/2 - 2 \cdot T1/2 - ((T4 - T1) - (T3 - T2))/2 = \\ &= (2 \cdot T2 - 2 \cdot T1 - ((T4 - T1) - (T3 - T2)))/2 = \\ &= (T2 + T2 - T1 - T1 - T4 + T1 + T3 - T2)/2 = \\ &= (T2 - T1 - T4 + T3)/2 = ((T2 - T1) + (T3 - T4))/2 \end{aligned}$$

Quindi:

$$\text{Offset} \approx ((T2 - T1) + (T3 - T4))/2$$

Meccanismo **three-way** di Sincronizzazione (1)

- Il master spedisce il proprio clock T1 nel messaggio di richiesta.
- Lo slave risponde mettendo nel messaggio di risposta il valore T1 ricevuto, il valore T2 dell'istante di ricezione, il valore T3 dell'istante di spedizione della risposta.
- Il master approssima il tempo di trasmissione dal master allo slave con la metà del round trip time

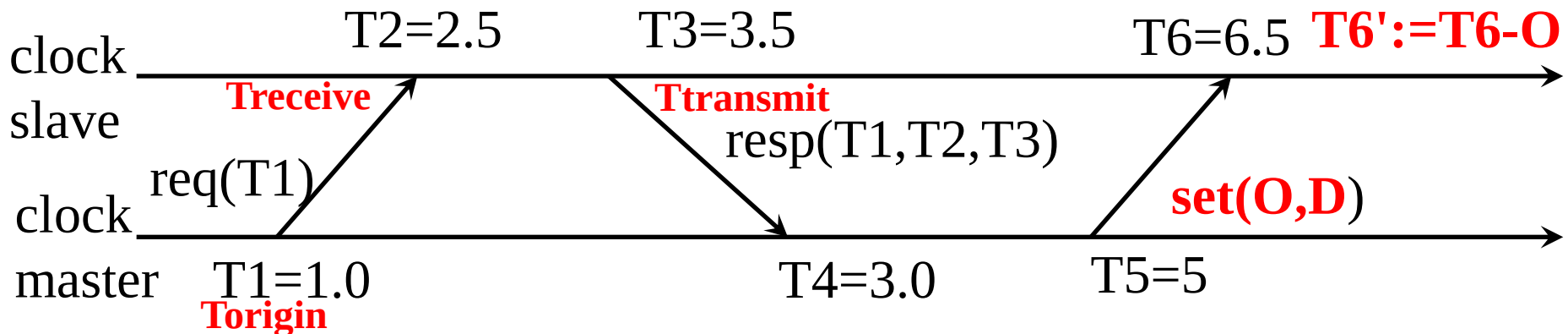
$$\text{Delay} \quad D = \text{RTT}/2 = ((T4 - T1) - (T3 - T2))/2 \quad (1)$$

- Il master approssima l'offset dell'orologio dello slave rispetto al master

$$\text{Offset} \quad O = ((T2 - T1) + (T3 - T4))/2 \quad (2)$$

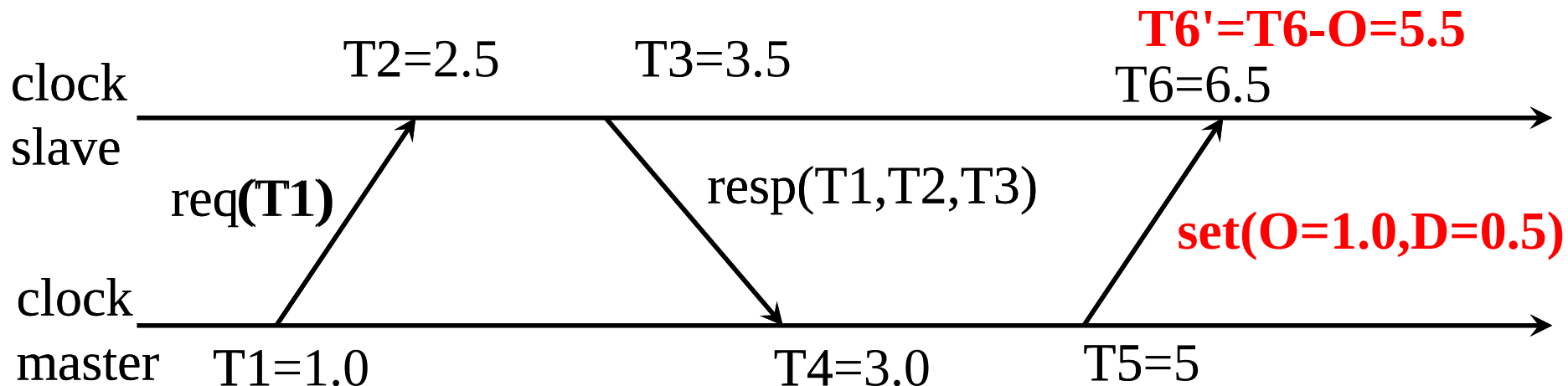
- Offset indica di quanto bisogna rallentare il clock dello slave (se $\text{RTT}=0$)
- Infine, il master invia l'ordine di modifica del clock dello slave con un messaggio contenente l'offset più il tempo di trasmissione
- Lo slave toglie al suo clock l'offset ricevuto

$$T_{\text{slave}}' := T_{\text{slave}} - O$$



Meccanismo **three-way** di Sincronizzazione (2)

- Esempio: supponiamo
 - che clock del master sia indietro di 1 secondo rispetto a clock di slave
 - che il tempo di trasmissione sia costante, circa 0.5 secondi
 - che il tempo $(T3-T2)$ prima di rispondere sia di 1 secondo
- allora
 - round trip time $RTT=((T4-T1)-(T3-T2))=(3.0-1.0)-(3.5-2.5)=2.0-1.0=1.0$
 - Delay $D=RTT/2=1.0/2=0.5$
 - Offset $O=((T2-T1)+(T3-T4))/2=((2.5-1.0)+(3.5-3.0))/2=(1.5+0.5)/2=1.0$
 - Offset indica di quanti secondi bisogna rallentare il clock dello slave
- All'istante $T5$ =Master manda messaggio $set(O=1.0, D=0.5)$
- All'istante $T6=6.5$ Slave riceve messaggio e modifica il proprio clock settandolo al nuovo valore $T6'=T6-O=6-1.0=5.5$



Problemi nella Sincronizzazione

Problemi degli algoritmi di sincronizzazione di orologi

- A causa dei diversi ritardi di trasmissione ogni processo non può avere una vista istantanea globale di tutti gli altri clock.
- Anche se tutti i clock fossero inizializzati contemporaneamente, essi non rimarrebbero sincronizzati a causa del drift nei rispettivi periodi
- La presenza di elementi malfunzionanti (faulty) distribuiti nel sistema, complica ulteriormente il raggiungimento di una visione unica del tempo

Sincronizzazione dei clock fisici 1/4

- Spesso non basta un ordinamento temporale non ambiguo, ma ha importanza il tempo vero in cui si verificano gli eventi. Per questi sistemi non è sufficiente il clock logico ma bisogna fare riferimento ai clock fisici e ai loro derivati software.
- Per distinguerli dai clock logici, che forniscono solo un ordinamento degli eventi, chiamiamo **clock fisici** i clock hardware o software **poiché forniscono [una approssimazione] del tempo reale.**
- Per ragioni di efficienza e ridondanza è auspicabile che nei sistemi complessi ci siano diversi clock fisici;
- Ne conseguono tre problemi:
 1. Definire un tempo ufficiale e i mezzi fisici per ottenerlo
 - UTC, Universal Coordinated Time
 2. Stabilire come sincronizzare gli orologi direttamente con i detentori del tempo ufficiale (sincronizzazione diretta)
 - WWV
 3. Stabilire come sincronizzare gli orologi tra loro
 - NTP, Network Time Protocol

Sincronizzazione dei clock fisici 2/4

1) Convenzione: Universal Coordinated Time - UTC

- 50 laboratori sparsi nel mondo che hanno orologi atomici basati su cesio 133. Periodicamente ogni laboratorio comunica al BIH(Bureau International de l'Heure di Parigi) quanti tick ha battuto il proprio orologio.
- BIH esegue la media di questi valori e produce il TAI, Tempo Atomico Universale.
- Il giorno del TAI dura circa 3 microsecondi in meno del giorno solare
- Il BIH risolve il problema inserendo del tempo di compensazione.
- Il sistema TAI compensato, prende il nome di UTC, Universal Coordinated Time, che è l'attuale standard internazionale.
- la sigla **UTC** è un compromesso tra l'inglese *Coordinated Universal Time* e il francese *Temps universel coordonné*¹,
- UTC è il fuso orario di riferimento da cui sono calcolati tutti gli altri fusi orari del mondo. Esso coincide, a meno di approssimazioni infinitesime, col tempo medio di Greenwich (*Greenwich Mean Time*, GMT).

Sincronizzazione dei clock fisici 3/4

2) Sincronizzazione diretta degli orologi

- Per mettere a disposizione l'UTC a quegli utenti che necessitano di un tempo preciso, esiste una stazione radio a onde corte detta **WWV** che emette un beep ad ogni secondo UTC.
- Per poter compensare i ritardi introdotti dalla propagazione del segnale nell'atmosfera bisogna conoscere con accuratezza la distanza tra trasmettitore e ricevitore per calcolare il ritardo di propagazione.

Sincronizzazione dei clock fisici 4/4

3) Sincronizzazione degli altri orologi

- Dati due processi c e c' , essi si diranno **sincronizzati al più** di delta al tempo T se:

$$|c(T) - c'(T)| < \text{delta}$$

cioè se i due processi raggiungeranno il valore del tempo T al più con delta secondi di ritardo l'uno dall'altro.

- HP: non possiamo collegarci direttamente ai server WWV quindi sfruttiamo la sincronizzazione tra orologi cercando di ottenere uno tra i due seguenti risultati:
 - "**sincronizzazione "interna" di clock**" : non è disponibile il tempo reale, ho più clock fisici paritetici e l'obiettivo è minimizzare la massima differenza esistente tra ogni coppia di clock, coordinandoli tra di loro a formare un clock (quasi) comune.
 - **sincronizzazione "esterna" di clock**: un clock sorgente fornisce il tempo reale e l'obiettivo degli altri clock è essere vicini a questo tempo sorgente il più possibile.
- Di seguito, ci occuperemo della sincronizzazione esterna cioè di come sincronizzare il clock di uno o più host con una sorgente esterna

Sincronizzazione esterna dei clock fisici

- Definiamo **time server process** il processo che si occupa della sincronizzazione. Ogni time server process può leggere il **clock hardware (HC)** dell'host al tempo **t**.
- Il **clock software (SC)** è implementato via software come:
$$SC(t) = HC(t) + A(t)$$

dove **A(t)** è la funzione di correzione dipendente dal tempo.
- Il problema è come definire la funzione di correzione.
- La presenza di malfunzionamenti nei sistemi (processori, clock, link di comunicazione) diminuisce l'efficacia dei vari algoritmi di sincronizzazione. Gli algoritmi devono continuare a funzionare anche quando un piccolo numero dei sistemi interessati alla sincronizzazione ha dei malfunzionamenti, anche quando i sistemi malfunzionanti sono i sorgenti del clock reale (o le reti usate per comunicare con questi clock).
- E qui interviene il protocollo NTP

Introduzione a NTP

- Per sincronizzare l'orario fra i server è disponibile il protocollo NTP (Network Time Protocol).
- Esistono molti server NTP pubblici utilizzabili per la sincronizzazione.
- NTP può sincronizzare gli orologi dei client al millisecondo, se i server sono nella stessa LAN, ed entro decine di millisecondi attraverso una WAN.
- I server NTP sono divisi in strati (**stratum**).
- I server NTP primari sono classificati come **stratum 1**; essi non devono essere usati per sincronizzare le macchine perché sono in numero esiguo.
- I server pubblici dello **stratum 2** sono disponibili per la sincronizzazione dei client e a loro volta si sincronizzano con i server pubblici stratum 1.
- Qui (<http://support.ntp.org/bin/view/Servers/StratumTwoTimeServers>) è disponibile una lista di server pubblici dello stratum 2.
- Anticipazione: configurando un servizio di directory, dovremo impostare i server Kerberos per interrogare tre server stratum 2 usando NTP.

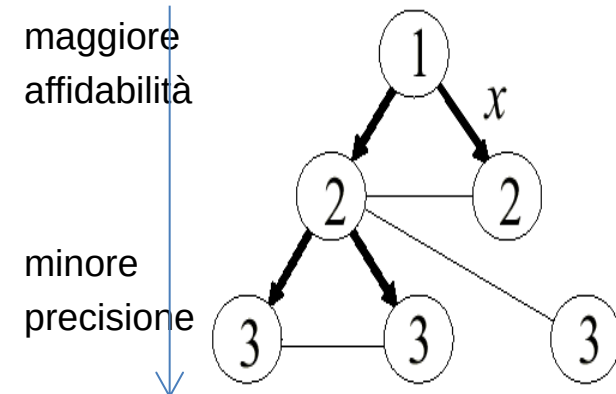
L'Infrastruttura di NTP (1)

(Network Time Protocol)

- Ogni computer è dotato di un orologio hardware scarsamente preciso che fornisce un tick ad intervalli non troppo costanti.
- Esistono però degli speciali ricevitori che consentono di ricevere il segnale orario da diverse sorgenti, come ad esempio il WWV o il GPS.
- A causa del costo e delle difficoltà trasmissive non è pensabile equipaggiare ogni computer con questi apparati.
- E' invece possibile equipaggiare alcuni computer con questi ricevitori facendoli poi agire come server primari del tempo.
- Altri computer si sincronizzeranno direttamente a questi server primari, agendo poi come server secondari per fornire il loro clock ad altri computer.
- Ulteriori computer si sincronizzeranno ai server secondari e così via, costruendo una infrastruttura di server di almeno due livelli ma anche di più.
- Tutti questi server costituiscono l'infrastruttura di NTP, organizzata in livelli detti **startum**.

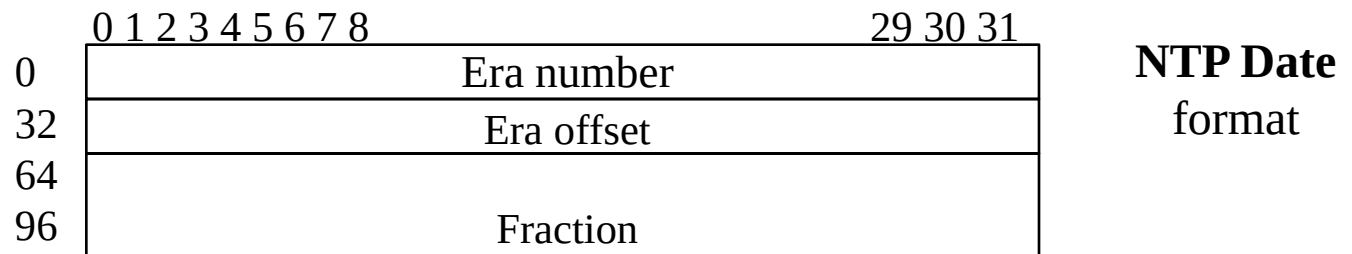
L'Infrastruttura di NTP (2)

- NTP organizza i suoi nodi ad albero:
- i server NTP primari, quelli che si sincronizzano direttamente a qualche fonte esterna (GPS, orologi atomici, fonti radio), si trovano alla radice, detta *stratum 1*.
- ai livelli successivi si trovano i server NTP secondari, terziari etc etc
- i client NTP sono computer che acquisiscono il clock dai server ma che non propagano la sincronizzazione ad altri computer.
- I client NTP possono collegarsi a server NTP di stratum 2 o successivi.
- Più si è in alto di livello e maggiore sarà la precisione del clock e maggiore l'affidabilità della sorgente poiché si tratterà di server importanti e ridondati.
- Per mantenere la precisione dei server è necessario mantenere un carico di lavoro basso per questi server, per cui un numero adeguato di server stratum 2 si collegano ad essi, in modo da poter distribuire a loro volta il tempo attraverso NTP direttamente ai client che ne facciano richiesta: l'utilizzo di server stratum 1 da parte di client è espressamente sconsigliato.
- Esistono 300 server stratum 1, 25000 stratum 2 e 85000 stratum 3.



Formato dei Dati NTP (1)

- L'attuale versione di NTP è la 4, definita principalmente nella RFC 5905 "Network Time Protocol Version 4: Protocol and Algorithms Specification" e con alcuni dettagli concernenti sicurezza definiti in altre RFC.
- NTP usa 3 diversi formati per rappresentare un istante, e sono tutti dati di tipo intero salvati in modo Big Endian (bit più significativi all'inizio (a sinistra negli schemi)): **NTP Date** format, **NTP Timestamp** format e **NTP Short** format.
- **NTP Date** format, ha 128 bit, è spezzato in due parti da 64 bit ciascuna, dove la prima parte rappresenta un istante espresso in secondi (interi) su un intervallo di 584 milioni di anni, la seconda parte a 64 bit specifica una aggiunta di una frazione di secondo, con una precisione di $0.5e-18$. La prima parte è suddivisa in due ulteriori parti, ciascuna da 32 bit: la prima parte (Era number) stabilisce una **Era** di 136 anni, la seconda parte (Era offset) stabilisce uno scostamento in secondi a partire dall'inizio dell'era specificata.
- NTP Date è il formato con cui NTP effettua internamente i calcoli sui tempi.



Formato dei Dati di NTP (2)

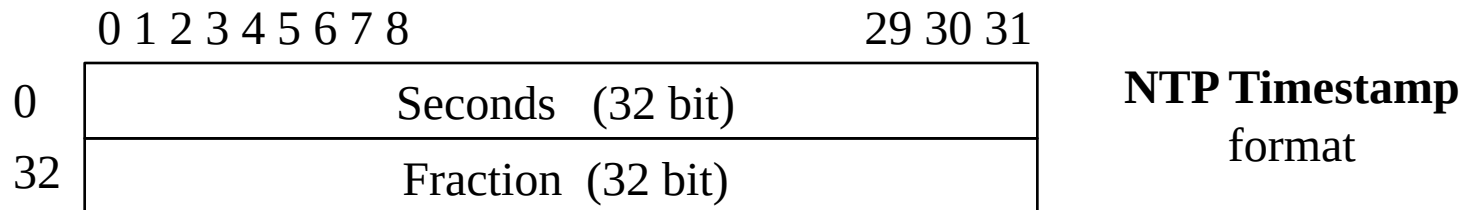
- Il formato **NTP Data** distingue il tempo in Ere, indicandole con l'indice numerico presente nel campo a 32 bit Era number.
- L'era attuale ha indice 0 e va dal 1 Gennaio 1900 fino al 8 Febbraio 2036 escluso.
- Le Ere precedenti hanno indice più piccolo (quindi negativo) mentre le Ere successive hanno indice maggiore di zero.

Vediamo alcune date interessanti

Date	NTP Era	NTP Timestamp Era Offset	Epoch (Rilevanza della data)
1 Jan -4712	-49	1,795,583,104	1st day Julian
1 Jan -1	-14	139,775,744	2 DC
1 Jan 0	-14	171,311,744	1 DC
1 Jan 1	-14	202,939,144	1 DC
4 Oct 1582	-3	2,873,647,488	Last day Julian
15 Oct 1582	-3	2,874,597,888	First day Gregorian
31 Dec 1899	-1	4,294,880,896	Last day NTP Era -1
1 Jan 1900	0	0	First day NTP Era 0
1 Jan 1970	0	2,208,988,800	First day UNIX
1 Jan 1972	0	2,272,060,800	First day UTC
31 Dec 1999	0	3,155,587,200	Last day 20 th Century
7 Feb 2036, 06:28:16	1	0	First second of Era 1
8 Feb 2036	1	63,104	Midnight of First day NTP Era 1

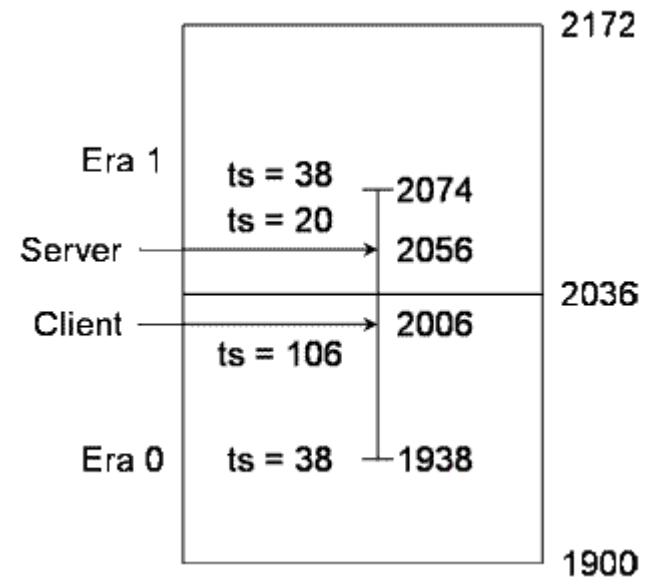
Formato dei Dati NTP (3)

- **NTP Timestamp** format, è un altro formato di più piccole dimensioni che viene usato in tutte le comunicazioni NTP per risparmiare banda.
 - Ha solo 64 bit, una metà dei quali usati per memorizzare i secondi e l'altra metà per memorizzare una frazione di secondi.
 - Non considera le Ere, quindi viene usato solo per incrementare e decrementare il clock all'interno di intervalli non eccessivamente grandi.
 - Inoltre, avendo solo 32 bit per la frazione di secondi, non consente una precisione elevatissima, ma non è un grosso limite poiché comunque trasmettendo in rete la precisione non la potremmo supportare poiché siamo soggetti alle fluttuazioni dei tempi di trasmissione.
-
- **NTP Timestamp è il formato usato per inserire marche temporali nei pacchetti trasmessi.**



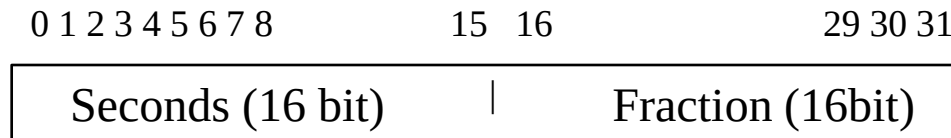
Obiettivo di NTP: aggiustare non settare

- NTP mira solo ad aggiustare il calcolo della funzione di correzione software $A(t)$ degli orologi software per allineare il tempo di un client con quello dei server.
- Il calcolo cioè non mira punta ad assegnare un valore definito all'orologio bensì a ricalcolare periodicamente un piccolo fattore di conversione rispetto al clock del server astraendo da quale sia la epoca considerata.
- Per poter aggiustare il valore del clock del client occorre che i due orologi (del client e del server) non siano troppo disallineati, in particolare devono avere due valori che non distano per più di una mezza Era (68 anni), altrimenti la correzione non è possibile.
- Quindi, per far funzionare NTP occorre che il clock di client e server inizialmente (cioè prima dell'entrata in azione del protocollo NTP) non siano temporalmente più distanti di 68 anni.
- **Inizializzazione degli orologi:** Il problema di come assegnare un valore iniziale non troppo sbagliato all'orologio deve essere risolto fuori da NTP. Ad esempio prendendo un valore iniziale da un file di configurazione.

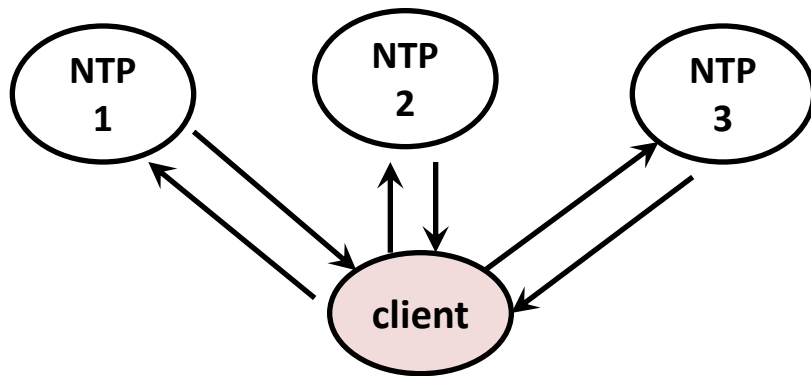


Formato dei Dati NTP (4)

- **NTP Short** format, è un altro formato di piccolissime dimensioni composto da soli 32 bit, metà dei quali dedicati a memorizzare secondi in forma intera, l'altra metà a memorizzare frazioni di secondo



NTP Short
format



Flusso dei messaggi NTP

- **Un client NTP** mantiene un elenco di server NTP a cui chiedere il clock.
- Ogni tanto, il client manda a ciascun server NTP una richiesta di clock, e ripete la richiesta più volte in modo da avere più campioni provenienti da ciascun server. Nella richiesta include il suo clock attuale ($T1 = T_{origin}$)
- Ciascun server NTP, che riceve la richiesta, risponde al client mettendo nel messaggio le marche temporali $T2 = T_{receive}$ e $T3 = T_{transmit}$.
- Il client riceve la risposta e si salva il tempo di ricevimento $T4$ per ciascuna risposta.
- Per ciascuna risposta il client calcola **Offset** e **Delay** secondo lo schema denominato **round trip**.
- Ottenuto un numero sufficiente di risposte, il client scarta le risposte "devianti".
- Il client usa una media pesata degli offset e stabilisce il nuovo tempo.
- Il client modifica la velocità del clock software.
- Ad intervalli variabili da 8 minuti a 36 ore la procedura si ripete.

Note sul Flusso dei messaggi NTP

NTP utilizza un messaggio il cui formato è descritto nella successiva slide.

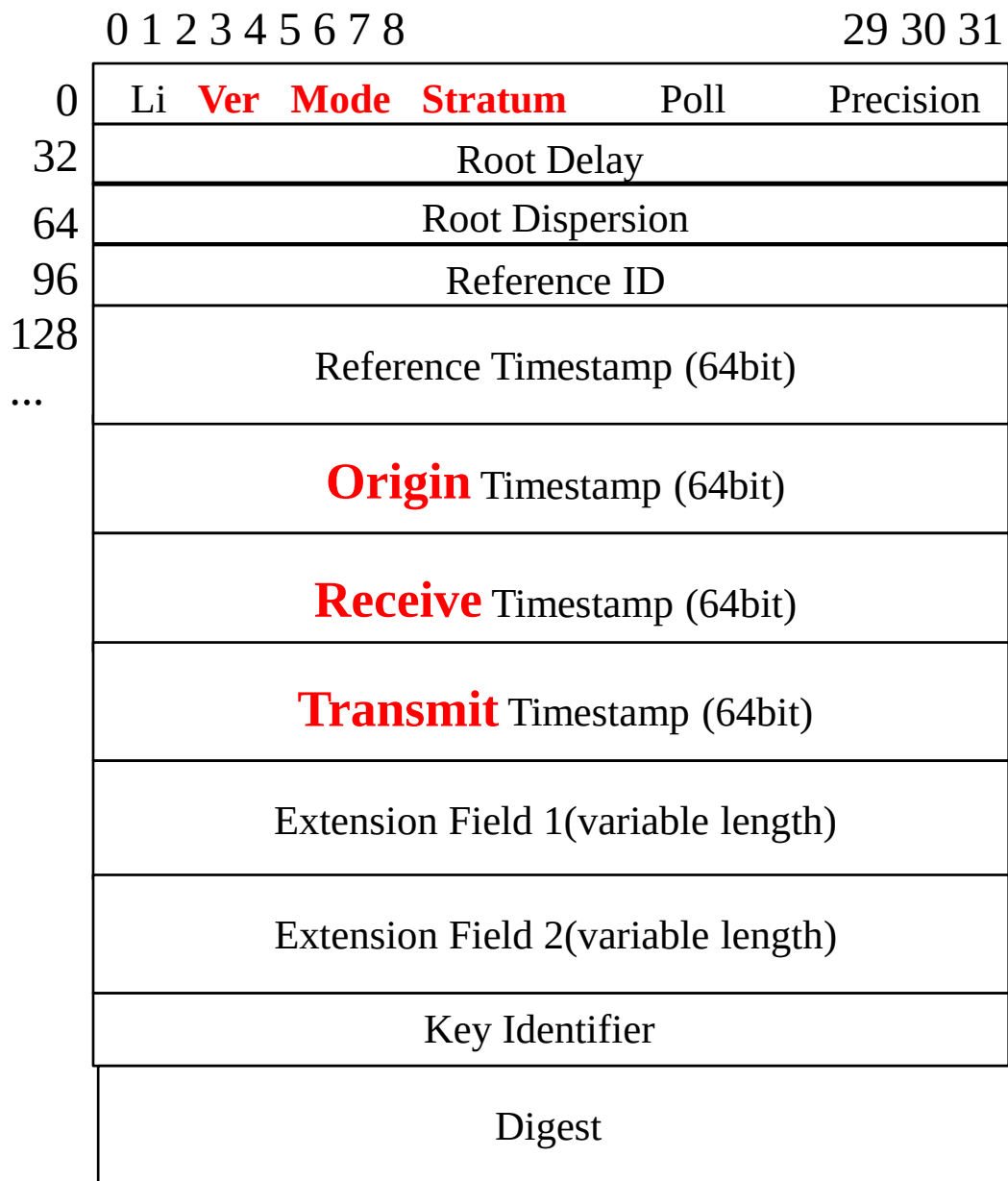
- Alcuni campi trasportano i citati T1, T2 e T3.
- Altri campi indicano il modo operativo del mittente e lo stratum del mittente.
- Il campo Reference Timestamp viene riempito dal client con il valore dell'ultimo clock settato, per dar modo al server di sapere cosa è accaduto della interazione precedente.
- I campi T2 e T3 vengono riempiti dal client con i valori ricevuti dal server nella precedente interazione.

Il Formato dei Pacchetti NTP (1)

Il messaggio NTP è
incapsulato in UDP
ed ha il seguente
formato

(vedere RFC 5905
per dettagli
<https://tools.ietf.org/html/rfc5905#page-12>)

I listener ascoltano
sulla porta UDP
predefinita 123



Il Formato dei Pacchetti NTP (2)

- **Leap Indicator (LI)** Codice a due bit di avviso per un leap second da inserire o cancellare dall'ultimo minuto del corrente giorno. Si possono avere i seguenti casi: 00 nessun avviso, 01 l'ultimo minuto ha 61 secondi, 10 l'ultimo minuto ha 59 secondi, 11 condizione di allarme (orologio non sincronizzato)
- **Version Number (VN)** Un intero a tre bit che indica il numero di versione di NTP/SNTP. Un valore pari a 3 indica NTPv3, 4 indica NTPv4.
- **Mode** Un intero a tre bit che indica la modalità della comunicazione, i cui valori sono definiti come segue: 0 riservato, 1 attiva simmetrica, 2 passiva simmetrica, 3 client, 4 server, 5 broadcast, 6 riservato per i messaggi di controllo NTP, 7 riservato per usi privati.
- **Stratum** Un intero di otto bit che indica lo stratum dell'orologio locale, i cui valori sono definiti come segue: 0 non specificato o non disponibile, 1 riferimento primario, 2-15 riferimento secondario (via NTP/SNTP), 16-255 riservato
 - Nella versione 4 del protocollo, lo stratum 0 è un particolare messaggio inviato dal server al client chiamato Kiss-of-Death, utilizzato per informare il client che qualcosa non è andato come doveva.
- **Poll Interval** Un intero di otto bit senza segno che indica il massimo intervallo tra due successivi messaggi in secondi come esponente del 2; indicando con PI il valore di questo campo, allora l'intervallo verrà calcolato come 2^{PI}
- **Precision** Un intero di otto bit con segno che indica la precisione dell'orologio locale in secondi come esponente del 2; indicando con PR il valore di questo campo, la precisione verrà calcolato come 2^{PR}

Il Formato dei Pacchetti NTP (3)

- **Root Delay** Un numero in virgola fissa a 32 bit con segno che indica il ritardo di roundtrip dalla sorgente di riferimento primario in secondi, con punto decimale fra i bit 15 e 16.
- **Root Dispersion** Un numero in virgola fissa a 32 bit senza segno che indica l'errore nominale relativo alla sorgente di riferimento primario, in secondi, con punto decimale tra i bit 15 e 16.
- **Reference Identifier** Una stringa di 32 bit che identifica la particolare sorgente di riferimento. In NTPv4, nel caso di stratum 1 è una stringa di quattro caratteri ASCII che identifica la fonte di sincronizzazione esterna (in stratum 0 rappresenta il messaggio spedito al client) mentre per stratum superiori è l'indirizzo IPv4 della sorgente di sincronizzazione.
- **Reference Timestamp** L'ora in cui è stato impostato o corretto per l'ultima volta l'orologio locale, nel formato NTP.
- **Originate Timestamp (T1)** L'ora in cui la richiesta ha lasciato il client per il server, nel formato NTP.
- **Receive Timestamp (T2)** L'ora in cui la richiesta è arrivata al server, nel formato NTP.
- **Transmit Timestamp (T3)** L'ora in cui la risposta ha lasciato il server per il client, nel formato NTP.

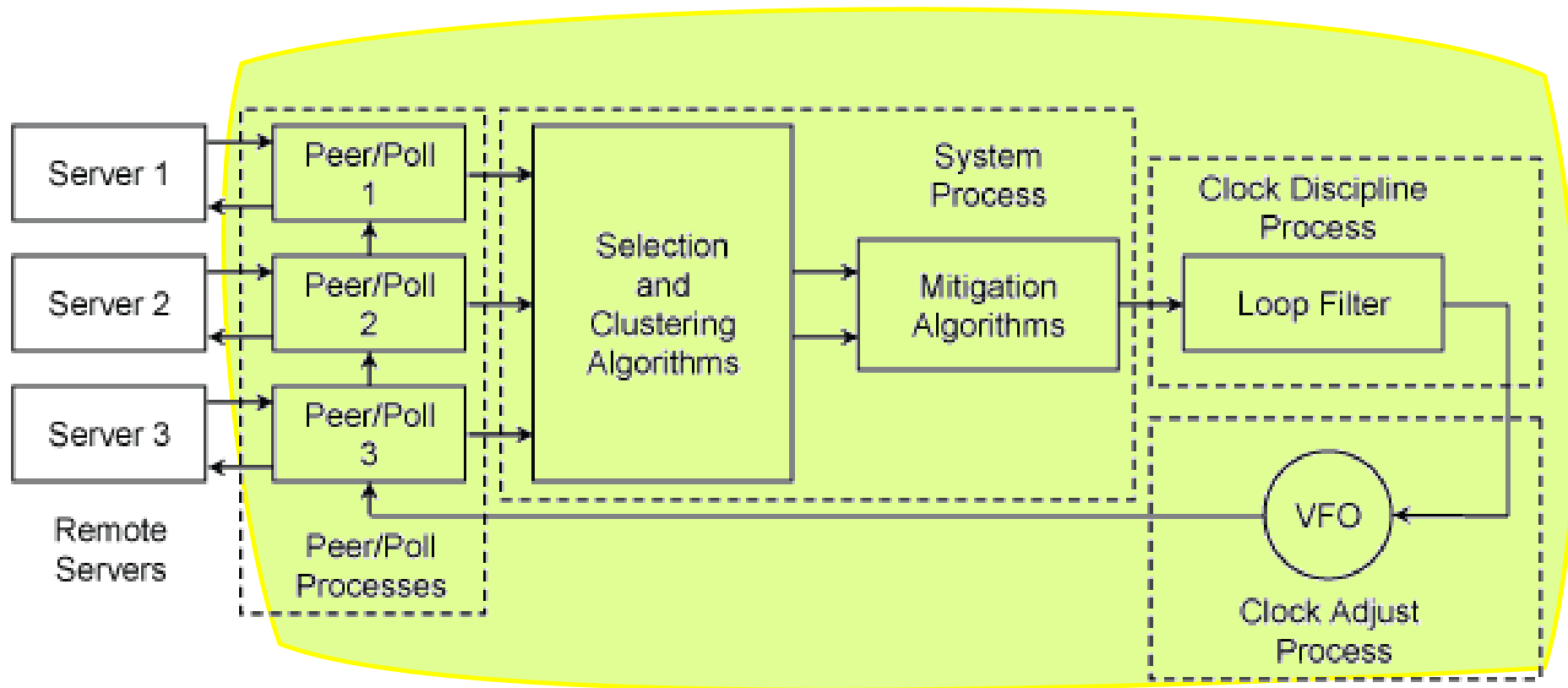
Domanda d'esame: Perché nel messaggio non c'è spazio per T4 ?

Varianti di funzionamento di NTP e modi delle entità

- NTP opera in modo diverso in funzione del ruolo (modo) delle entità coinvolte e manda pacchetti diversi (vedi campo mode) in funzione del ruolo del mittente:
- Esistono 3 varianti di funzionamento di NTP: client/server, symmetric e broadcast.
- Nella variante **client/server**, un client manda un pacchetto di modo 4 ad un server che risponde con pacchetti di modo 5. Il server fornisce sincronizzazione a più client. Il server può avere un ricevitore WWV o GPS o Galileo. Il server non accetta sincronizzazioni da altri.
- Nella variante **simmetrica**, una entità svolge entrambi i ruoli, ruolo di client rispetto ad alcuni server da cui riceve la sincronizzazione, e ruolo di server verso altri client a cui propaga la sincronizzazione. La comunicazione, comunque, è di tipo uno ad uno, sfruttando pacchetti con modo 1 e 2.
- ~~• Nella variante **broadcast**, un server manda pacchetti in broadcast verso tanti clienti. Alcuni di questi clienti possono temporaneamente trovarsi in una condizione temporanea effimera (ephemeral) durante la quale operano per determinare il ritardo di propagazione dei pacchetti attraverso quel server e per effettuare le necessarie procedure di sicurezza (Autokey security protocol).~~
- **NB: La topologia della rete del tempo deve essere realizzata per evitare loop e per ridurre al minimo i server da attraversare per ottenere la sincronizzazione dagli stratum 1. Il Dynamic Server Discovery service di NTP forma la topologia.**

Architettura Software e Algoritmo di NTP (1)

- In ciascun nodo NTP (client o server) eseguono alcuni processi che svolgono funzioni specifiche. Tali processi sono identificabili nel seguente schema:



Per descrizione vedi pagina successiva

Architettura Software e Algoritmi di NTP (2)

- Ad intervalli variabili tra 8 e 36 ore, i processi Pool inviano messaggi RTP ed i Peer li ricevono scartando quelli malformati e che non superano controlli di sicurezza.
- I pacchetti contengono le tre informazioni basiche Origin Received e Transmit Timestamps e permettono ai Peer di calcolare, **per ciascun server di provenienza**, le due grandezze fondamentali, Offset e Delay nello stesso modo giù visto nell'algoritmo di Berkeley: $\text{Offset} = ((T2 - T1) + (T3 - T4)) / 2$ $\text{Delay} = (T4 - T1) - (T3 - T2)$
- Gli algoritmi di selezione e clustering scartano i pacchetti con ritardi anomali e chiedono, ad un algoritmo di mitigazione, di mantenere i soli pacchetti più adatti a calcolare il tempo, in base a criteri sia statistici che relativi ai server di provenienza.
- Infatti, l'algoritmo di selezione e clustering organizza la topologia, mantenendo nel gruppo dei propri server immediati solo quelli che hanno fornito i pacchetti migliori, in particolare quelli più vicini e con il cammino più corto fino agli stratum 1.
- Nel frattempo l'algoritmo di mitigazione ha costruito una media pesata tra i diversi Offset e Delay e la usa per ordinare al processo Clock Discipline di modificare lentamente la velocità del clock software, modificando la funzione $A(t)$ di aggiustamento del clock. L'aggiustamento viene poi fisicamente realizzato dal Clock Adjust. che pilota l'oscillatore a frequenza variabile VFO.

Cenni su sicurezza in NTP

- I principali problemi di sicurezza in NTP sono dovuti principalmente a tentativi di
 - fingersi un server e mandare clock errati ad un client
 - spedire una marea di messaggi ad un server per sovraccaricarlo
- NTP struttura la sicurezza adottando più tecniche, anche contemporaneamente, in funzione della variante di funzionamento, poiché a diverse varianti corrispondono entità che funzionano in modo diverso.
- Nella variante client/server il problema principale è identificare il server, e questo può essere fatto con diverse tecniche, qui elencati in ordine crescente di sicurezza e anche di livello architetturale:
 - random nonce per individuazione di duplicati
 - message digest
 - autokey sequence
 - certificati digitali
- Perché allora non usare sempre il maggior meccanismo di sicurezza? I certificati digitali, rilasciati al server da autorità di certificazione, hanno il problema di essere computazionalmente onerosi, quindi si preferisce non autenticare tutti i messaggi ma solo alcuni ogni tanto.
- Per le altre varianti i problemi sono esacerbati, ad esempio nella variante simmetrica occorre identificare tutte le entità, e questo aumenta i problemi.

NTP su client Linux (1)

- In una macchina Linux posso sincronizzare il clock usando NTP in due momenti diversi:
 - subito dopo il boot della macchina, lanciando una tantum lo script **ntpdate**
 - mentre il sistema è in esecuzione, mantenendo in esecuzione il demone **ntpd**
- Il comando **ntpdate** deve operare come root ed è un po' grezzo, si collega ad uno o più server NTP passati come argomenti e li interroga mediante il protocollo NTP per conoscere l'offset del proprio clock rispetto al server. Il comando applica alcuni degli algoritmi di selezione e filtering dei campioni ricevuti e determina il migliore.
- Dopo avere individuato l'aggiustamento da effettuare, il comando ntpdate setta l'orologio in due modi diversi:
 - Se l'errore è maggiore di 0.5 secondi, ntpdate setta direttamente l'orologio invocando la funzione `settimeofday()`
 - Se invece l'errore è minore di 0.5 secondi, ntpdate usa la funzione `adjtime` che modifica gradatamente la velocità del clock.
- Il comando ntpdate è meno accurato dell'intero protocollo NTP. Si consiglia perciò, dopo il primo settaggio al boot con ntpdate, di **lanciare il demone ntpd** che monitorerà costantemente il clock, con maggior accuratezza e minor carico.
- In alternativa, si può configurare il sistema (cron) per lanciare ntpdate ad intervalli regolari.

NTP su client Linux (2)

- Il comando **ntpq** serve per interrogare il demone locale di NTP. Lanciando semplicemente il comando `ntpq -p` viene visualizzato lo stato dei server attualmente utilizzati dal demone `ntpd` locale.
- il comando `ntpstat` visualizza informazioni sullo stato di sincronizzazione del tempo fornito da NTP.

Bibliografia su sincronizzazione e NTP

- Specifiche ufficiali NTP: RFC 5905 - Network Time Protocol Version 4: Protocol and Algorithms Specification. <https://tools.ietf.org/html/rfc5905>
- Dettagli più comprensibile su NTP, <https://www.eecis.udel.edu/~mills/time.html>
- Clock Discipline <https://www.eecis.udel.edu/~mills/ntp/html/discipline.html>
solo per chi si vuole male
- ntpdate command: <http://doc.ntp.org/4.1.1/ntpdate.htm>
- Per chi volesse sincronizzare il clock direttamente da satellite, disponendo di un GPS (o analoghi) <http://www.timetoolsglobal.com/information/gps-ntp-server/>